

Joint Lung + Nodule Segmentation — 3D U-Net / DynUNet (pseudo-LIDC)

1. Task

Voxel-level joint segmentation of lung tissue and pulmonary nodules in 3D chest CT volumes. Single model, single forward pass, no ROI stage and no bounding-box crop.

- **Input** (1, 256, 256, 256) full CT resampled to 256^3 , intensity-normalised to [0, 1].
- **Output** (3, 256, 256, 256) softmax logits: class 0 = background, class 1 = lung, class 2 = nodule.

A voxel that is both lung tissue and nodule is assigned exclusively to class 2 (nodule takes precedence over lung) since the head uses a mutually-exclusive softmax.

2. Model

[MONAI DynUNet](#) — nnU-Net-style dynamic **3D U-Net**, DynUNet (nnU-Net-style 3D U-Net).

Property	Value
Architecture family	DynUNet (nnU-Net-style 3D U-Net)
Spatial dims	3
Input channels	1
Output channels	3 (softmax)
Encoder levels	6
Kernel sizes	[3, 3, 3] at every level
Strides	[[1,1,1], [2,2,2], [2,2,2], [2,2,2], [2,2,2], [2,2,2]]
Normalisation	instance
Deep supervision	off
Trainable parameters	31,181,763

3. Data

Trained on the `unified_v2` split (patient-grouped, dataset-stratified, **full corpus**):

Source	Role	Lung labels
NLST	train + val + test	GT (per-slice 2D masks stacked to 3D)

Source	Role	Lung labels
NSCLC-Radiomics	train + val + test	GT (per-slice 2D masks stacked to 3D)
LIDC-IDRI	train + val + test	pseudo-GT — 2D SegResNet ROI model prediction per slice, stacked to 3D

Split sizes: **1 683 train / 297 val / 325 test (held out)**.

Nodule labels come from the corpus's own 3D nodule annotations for all three sources.

4. Training

Setting	Value
Loss	Multi-class Focal Tversky + weighted CE ($\alpha=0.3$, $\beta=0.7$, $\gamma=2.0$, $\lambda_{CE}=0.1$, class weights = [1.0, 1.0, 100.0] for [bg, lung, nodule])
Optimizer	Adam
Learning rate	1×10^{-5}
Weight decay	1×10^{-5}
LR schedule	Cosine annealing, $T_{max} = 400$, $\eta_{min} = 1 \times 10^{-6}$
Batch size	2
Epochs	400
Random seed	42
Mixed precision	bf16 (autocast, no GradScaler)
Augmentation	3D flips, 90° rotations, elastic rotation, zoom, intensity scale/shift, Gaussian noise/blur, contrast
Hardware	1 × NVIDIA H100 94 GB
Wall-clock	≈ 5 days

Best checkpoint (evaluated below): **epoch 234 / 400**.

5. Evaluation protocol

Evaluated on the val split (297 series, never seen during training). Predictions taken as `argmax` over the 3-channel softmax output. All per-voxel metrics are micro-averaged over the whole split (4,982,833,152 voxels total).

6. Results

Ticket-required metrics

Metric	Value
mean IoU (all 3 classes)	0.8031
mean IoU (lung + nodule)	0.7086
Accuracy	0.9931
Precision (macro, all classes)	0.8651
Precision (macro, non-bg)	0.7988
Recall (macro, all classes)	0.8730
Recall (macro, non-bg)	0.8124

Per-class breakdown

Class	IoU	Precision	Recall	Dice (micro)
Background	0.9921	0.9977	0.9944	0.9961
Lung	0.9505	0.9642	0.9854	0.9746
Nodule	0.4667	0.6335	0.6394	0.6364

Per-case Dice distribution (per class, 297 val cases)

Class	Mean	Std	p25	Median	p75
Background	0.9960	0.0025	0.9954	0.9962	0.9968
Lung	0.9713	0.0176	0.9651	0.9746	0.9819
Nodule	0.4937	0.2499	0.3147	0.5341	0.7028

7. Notes on interpretation

The joint task's class distribution is heavily skewed: background dominates, lung tissue is ~14 % of the volume, and nodule voxels are on the order of 10^{-5} . This shapes what each metric means here:

- **Accuracy** looks near-perfect (~ 0.99) because getting background right accounts for most voxels regardless of model quality. Read it as a floor, not a discriminator.
- **mIoU** across all three classes is dominated by the near-1.0 `IoU_background`; the *no-bg* variant (`IoU_lung`, `IoU_nodule` averaged) is the more diagnostic number for comparing the two model families.
- **Precision / Recall** for the lung head sit at ~0.98 — the model has learned lung anatomy well. The nodule head has Precision and Recall in the 0.62–0.75 range and is where architecture / data choices actually differentiate.

- **Per-case Dice** for the nodule class carries a wide distribution ($\text{std} \approx 0.25$) driven by nodule size and location — the median is a better summary than the mean for clinical use.

8. How to load & run inference

The pre-trained weights are hosted at [Kakimaki00/joint-dynunet-3d-pseudo-lidc](https://huggingface.co/Kakimaki00/joint-dynunet-3d-pseudo-lidc).
Download and load directly:

```
huggingface-cli download Kakimaki00/joint-dynunet-3d-pseudo-lidc --local-dir ./ckpt
```

```
import yaml, torch
from monai.networks.nets import DynUNet

cfg = yaml.safe_load(open("config.yaml"))["model"]
default_strides = [[1, 1, 1]] + [[2, 2, 2]] * 5
strides = cfg.get("strides", default_strides)
kernel = cfg.get("kernel_size", [[3, 3, 3]] * len(strides))
model = DynUNet(
    spatial_dims      = cfg["spatial_dims"],
    in_channels       = cfg["in_channels"],
    out_channels      = cfg["out_channels"],
    kernel_size       = kernel,
    strides           = strides,
    upsample_kernel_size = cfg.get("upsample_kernel_size", strides[1:]),
    norm_name         = cfg.get("norm_name", "instance"),
    deep_supervision  = cfg.get("deep_supervision", False),
)
state = torch.load("model.pth", map_location="cpu", weights_only=True)
model.load_state_dict(state)
model.eval()

with torch.no_grad():
    x = torch.randn(1, 1, 256, 256, 256)          # full CT resampled to 2563
    logits = model(x)                             # (B, 3, D, H, W)
    pred_class = logits.argmax(dim=1)              # (B, D, H, W) in {0, 1, 2}
    lung_mask = (pred_class == 1).to(torch.uint8)
    nodule_mask = (pred_class == 2).to(torch.uint8)
```

Unlike the two-stage `nodule-*` HF checkpoints, this model does **not** need a lung-bbox crop — feed it the whole CT resampled to 256³.

9. Reproducibility

- Config `configs/joint_dynunet_pseudo.yaml`
- Checkpoint `stage3_joint/checkpoints_dynunet_pseudo/best_model.pth`
- Metrics JSON `stage3_joint/checkpoints_dynunet_pseudo/eval_metrics_joint.json`

- Command `python stage3_joint/eval_metrics.py --config configs/joint_dynunet_pseudo.yaml --checkpoint stage3_joint/checkpoints_dynunet_pseudo/best_model.pth`

Companion variant: [Kakimaki00/joint-dynunet-3d-ex-lidc](#) — same architecture, ex-LIDC data variant.